



RELATÓRIO

Módulo 11

Orientação a Objetos Avançado

CURSO TÉCNICO DE PROGRAMAÇÃO E GESTÃO DE SISTEMAS INFORMÁTICOS

Professor : Breno Sousa

Nome do Aluno: André Portela/Alexandre Afonso

Nº Aluno: L'2490/L'2454

23/10/2025

O relatório encontra-se em condições para ser apresentado

Ciclo de Formação 2023/2026

Ano Letivo 2024/2025

ÍNDICE

1ª Agradecimentos

2ª Introdução

3ª Plataforma Utilizada

4ª Melhores Partes do Código

5ª Conclusão

ÍNDICE DE FIGURAS

Figura 1- Código

Figura 2- Código

Figura 3- Código

Figura 4- Código

Figura 5- Código

CAPÍTULO I — AGRADECIMENTOS

Os alunos gostariam de expressar o seu sincero agradecimento ao professor Breno pela sua constante disponibilidade e dedicação. Ao longo de todo o tempo, demonstrou sempre uma enorme vontade em ajudar, esclarecendo dúvidas, oferecendo apoio e incentivando cada um de nós a dar o melhor de si.

A sua paciência, profissionalismo e simpatia tornaram o processo de aprendizagem muito mais agradável e enriquecedor.

CAPÍTULO II — INTRODUÇÃO

Este relatório detalha a criação de um sistema de gestão de um hotel em Python, desenvolvido como parte do Módulo 11. O objetivo foi empregar os princípios da Programação Orientação a Objetos Avançado , abrangendo heranças, polimorfismo e classes abstratas.

O sistema administra quartos de várias categorias (simples e luxo) e as reservas dos hóspedes, automatizando tarefas como check-in, cálculo de estadias e gerenciamento de funcionários. A aplicação foi criada em módulos distintos, o que garante a organização e simplifica a manutenção do código.

CAPÍTULO III — Plataforma Utilizada

Para o desenvolvimento deste projeto, os alunos optaram por utilizar o *Visual Studio Code* como ambiente principal de programação, devido à sua versatilidade e facilidade de utilização. Com o objetivo de permitir o trabalho colaborativo de forma simultânea, decidiram instalar a extensão *Code Together*, que possibilita a edição simultânea do código por vários utilizadores. Desta forma, todos os membros do grupo puderam contribuir de forma mais eficiente, partilhando ideias, corrigindo erros e desenvolvendo diferentes partes do projeto em conjunto.

Capítulo IV – Melhores Partes do Código

```
1  from abc import ABC, abstractmethod
2
3  class Pessoa(ABC):
4      def __init__(self, nome, idade):
5          self._nome = nome
6          self._idade = idade
7
8      @property
9      def nome(self):
10         return self._nome
11
12     @nome.setter
13     def nome(self, novo_nome):
14         self._nome = novo_nome
15
16     @property
17     def idade(self):
18         return self._idade
19
20     @idade.setter
21     def idade(self, nova_idade):
22         if nova_idade < 0:
23             print("Idade inválida - não pode ser negativa")
24         else:
25             self._idade = nova_idade
26
27     @abstractmethod
28     def mostrar_informacoes(self):
```

Figura 1- Código



```
3 class Funcionario(Pessoa):
4     def __init__(self, nome, idade, salario):
5         super().__init__(nome, idade)
6         self.salario = salario
7
8     def mostrar_informacoes(self):
9         return f"Nome: {self.nome}, Idade: {self.idade}, Salário: {self.salario}"
10
11 class Hospede(Pessoa):
12     def __init__(self, nome, idade, dias_estadia, quarto=None):
13         super().__init__(nome, idade)
14         self.dias_estadia = dias_estadia
15         self._quarto = quarto
16         if quarto:
17             quarto.ocupado = True
18
19     @property
20     def quarto(self):
21         return self._quarto
22
23     @quarto.setter
24     def quarto(self, quarto):
25         if quarto is not None and not quarto.ocupado:
26             self._quarto = quarto
27             quarto.ocupado = True
28             print("Quarto atribuído com sucesso!")
29         else:
30             print("Quarto inválido ou ocupado")
```

Figura 2- Código

```
1 from funcionario hospede import Funcionario, Hospede
2
3 class Rececionista(Funcionario):
4     def __init__(self, nome, idade, salario, turno):
5         super().__init__(nome, idade, salario)
6         self._turno = turno
7
8     @property
9     def turno(self):
10         return self._turno
11
12     @turno.setter
13     def turno(self, turno):
14         turnos_validos = ["manhã", "tarde", "noite"]
15         if turno.lower() in turnos_validos:
16             self._turno = turno
17         else:
18             print("Turno inválido. Use: manhã, tarde ou noite")
19
20     def mostrar_informacoes(self):
21         return f"Nome: {self.nome}, Idade: {self.idade}, Salário: {self.salario}, Turno: {self.turno}"
22
23     def registrar_hospede(self, hospede, lista_hospedes):
24         lista_hospedes.append(hospede)
25         print(f"Hóspede {hospede.nome} registrado com sucesso!")
26
```

Figura 3- Código



```
26
27     def listar_hospedes(self, lista_hospedes):
28         print("=== Lista de Hóspedes ===")
29         for hospede in lista_hospedes:
30             print(hospede.mostrar_informacoes())
31
32     class Gerente(Funcionario):
33     def __init__(self, nome, idade, salario):
34         super().__init__(nome, idade, salario)
35         self.bonus = 0.1 # 10% de bônus
36
37     def mostrar_informacoes(self):
38         salario_total = self.salario + (self.salario * self.bonus)
39         return f"Nome: {self.nome}, Idade: {self.idade}, Salário: {self.salario}, Bônus: {self.bonus*100}%, Total: {salario_total}"
40
41     def gerar_relatorio(self, funcionarios):
42         print("=== Relatório de Funcionários ===")
43         for funcionario in funcionarios:
44             print(funcionario.mostrar_informacoes())
45
```

Figura 4- Código

```
opcao = input("\nEscolha uma opção (1-5): ")
wait_n_clear()
match opcao:
    case "1":
        print("\n=== Registro de Hóspede ===")
        nome = input("Nome: ")
        idade = int(input("Idade: "))
        dias = int(input("Dias de estadia: "))
        wait_n_clear()
        print("\nQuartos disponíveis:")
        for quarto in quartos:
            if not quarto.ocupado:
                print(quarto.mostrar_informacoes())
        wait_n_clear()
        try:
            num_quarto = int(input("Número do quarto: "))
            quarto_escolhido = None
            for quarto in quartos:
                if quarto.numero == num_quarto and not quarto.ocupado:
                    quarto_escolhido = quarto
                    break
            if quarto_escolhido:
                hospede = Hospede(nome, idade, dias, quarto_escolhido)
                recepcionista.registrar_hospede(hospede, hospedes)
                print(f"Conta total: € {hospede.calcular_conta()}")
            else:
                print("Quarto não disponível!")
                wait_n_clear()
        except ValueError:
            print("Número de quarto inválido!")
```

Figura 5- Código

Capítulo V - CONCLUSÃO

Em conclusão, o desenvolvimento deste sistema de gestão de um hotel permitiu aplicar com sucesso os principais conceitos de Orientação a Objetos Avançado estudados no módulo. O projeto demonstrou na prática a utilidade das heranças simples e múltipla, do polimorfismo e das classes abstratas.

